

数学建模 非线性规划 3.1 3.2 3.3 3.4 题报告

2351114 大数据 朱俊泽

3.1 题目

3.1 某工厂向用户提供发动机,按合同规定,其交货数量和日期是:第一季度末交 40 台,第二季末交 60 台,第三季末交 80 台。工厂的最大生产能力为每季 100 台,每季的生产费用是 $f(x) = 50x + 0.2x^2$ (元),此处 x 为该季生产发动机的台数。若工厂生产得多,多余的发动机可移到下季向用户交货,这样,工厂就需支付存储费,每台发动机每季的存储费为 4 元。问该厂每季应生产多少台发动机,才能既满足交货合同,又使工厂所花费的费用最少(假定第一季度开始时发动机无存货)?

3.1.1 建模

题目要求工厂用料成本最小化,同时满足以下条件

- 工厂每季度需要生产 40 台第一季末未交、60 台第二季末未交、80 台第三季末未交的机器。

- 每台机器的生产能力为每季度 100 台。
- 每台机器的生产成本由函数 $f(x)=50x+0.2x^2$ 给出,其中 x 是机器数量
- 每季度末的机器数量需要满足题目给定的未交数量。
- 工厂希望用料成本最小,同时满足每季度末未交机器数量。

这是一个非线性的规划问题,于是我们关注他的目标函数和约束条件。最小化成本函数就可以使用 python 里面的 optimize 方法来做。

目标函数:

设 y_1, y_2, y_3 分别为第一季度、第二季度、第三季度的生产机器数量。

s_1, s_2 第一季度末、第二季度末的库存量(即未交付的发动机数量)

总成本为所有季度的生产成本之和:

$$(50(y_1 + y_2 + y_3) + 0.2(y_1^2 + y_2^2 + y_3^2)) + 4S_1 + 4S_2$$

目标是使总成本最小化。

约束条件:

第一季度末上交 40 台至少:

$$y_1 \geq 40$$

第二季度末至少上交了 60 台,也就是第一季度剩余加上第二季度生产的机器数量。

$$y_1 + y_2 \geq 100$$

第三季度末上交 80 台，也就是第二季度的剩余加上第三季度的生产。

$$y_1 + y_2 + y_3 \geq 180$$

还有每个季度的生产上限和非负限制。

$$100 \geq y_1 \geq 0$$

$$100 \geq y_2 \geq 0$$

$$100 \geq y_3 \geq 0$$

3.1.2 代码

```
import numpy as np
from scipy.optimize import minimize

# 定义目标函数：总成本（生产成本 + 存储成本）
def objective(vars):
    x1, x2, x3 = vars
    # 计算库存
    s1 = x1 - 40 # 第一季度末库存
    s2 = s1 + x2 - 60 # 第二季度末库存
    # 生产成本
    cost1 = 50 * x1 + 0.2 * x1**2
    cost2 = 50 * x2 + 0.2 * x2**2
    cost3 = 50 * x3 + 0.2 * x3**2
    # 存储成本
    storage_cost = 4 * s1 + 4 * s2
    # 总成本
    return cost1 + cost2 + cost3 + storage_cost

# 定义约束条件
constraints = [
    {'type': 'ineq', 'fun': lambda vars: vars[0] - 40}, # x1 = 80
    {'type': 'ineq', 'fun': lambda vars: (vars[0] ) + vars[1] - 100 }, # s1 + x2 = 120
    {'type': 'eq', 'fun': lambda vars: (vars[0] + vars[1] ) + vars[2] - 180}, # s2 + x3 = 160
    {'type': 'ineq', 'fun': lambda vars: 100 - vars[0]}, # x1 <= 100
    {'type': 'ineq', 'fun': lambda vars: 100 - vars[1]}, # x2 <= 100
    {'type': 'ineq', 'fun': lambda vars: 100 - vars[2]}, # x3 <= 100
]

# 初始猜测值
initial_guess = [80, 50, 50] # x1, x2, x3

# 变量的非负约束
bounds = [(0, None), (0, None), (0, None)] # x1, x2, x3 >= 0

# 求解优化问题
```

```

result = minimize(objective, initial_guess, method='SLSQP', bounds=bounds, constraints=constraints)
# 输出结果
if result.success:
    x1, x2, x3 = result.x
    s1 = x1 - 40
    s2 = s1 + x2 - 60
    total_cost = result.fun
    print(f"优化成功! ")
    print(f"第一季度生产数量 x1: {x1:.2f} 台")
    print(f"第二季度生产数量 x2: {x2:.2f} 台")
    print(f"第三季度生产数量 x3: {x3:.2f} 台")
    print(f"第一季度末库存 s1: {s1:.2f} 台")
    print(f"第二季度末库存 s2: {s2:.2f} 台")
    print(f"第一季度生产成本: {50 * x1 + 0.2 * x1**2:.2f} 元")
    print(f"第二季度生产成本: {50 * x2 + 0.2 * x2**2:.2f} 元")
    print(f"第三季度生产成本: {50 * x3 + 0.2 * x3**2:.2f} 元")
    print(f"存储成本: {4 * s1 + 4 * s2:.2f} 元")
    print(f"总成本: {total_cost:.2f} 元")
else:
    print("优化失败! ")
    print(result.message)

```

3.1.3 结果

第一季度生产数量 x1: 50.00 台

第二季度生产数量 x2: 60.00 台

第三季度生产数量 x3: 70.00 台

第一季度末库存 s1: 10.00 台

第二季度末库存 s2: 10.00 台

第一季度生产成本: 3000.00 元

第二季度生产成本: 3720.00 元

第三季度生产成本: 4480.00 元

存储成本: 80.00 元

总成本: 11280.00 元

检验约束条件:

$$y_1 \geq 40$$

50 ≥ 40。满足

$$y_1 + y_2 \geq 100$$

50+60=110>=100。满足

y1 + y2 + y3 ≥ 180

50+60+70=180=180。满足

同时也满足非负数约束和生产上限约束。

3.2 题目

3.2 用 Matlab 的非线性规划命令 fmincon 求解飞行管理问题的模型二。

例 3.10(本题选自 1995 年全国大学生数学建模竞赛 A 题) 在约 10000m 高空的某边长 160km 的正方形区域内,经常有若干架飞机水平飞行。区域内每架飞机的位置和速度向量均由计算机记录其数据,以便进行飞行管理。当一架欲进入该区域的飞机到达区域边缘时,记录其数据后,要立即计算并判断是否会与区域内的飞机发生碰撞。如果会碰撞,则应计算如何调整各架(包括新进入的)飞机飞行的方向角,以避免碰撞。现假定条件如下:

- (1) 不碰撞的标准为任意两架飞机的距离大于 8km;
- (2) 飞机飞行方向角调整的幅度不应超过 30°;
- (3) 所有飞机飞行速度均为 800km/h;
- (4) 进入该区域的飞机在到达区域边缘时,与区域内飞机的距离应在 60km 以上;
- (5) 最多需考虑 6 架飞机;
- (6) 不必考虑飞机离开此区域后的状况。

请你对这个避免碰撞的飞行管理问题建立数学模型,列出计算步骤,对以下数据进行计算(方向角误差不超过 0.01°),要求飞机飞行方向角调整的幅度尽量小。

设该区域 4 个顶点的坐标为 (0,0),(160,0),(160,160),(0,160)。飞行记录数据见表 3.7。

表 3.7 飞行记录数据

飞机编号	横坐标 x	纵坐标 y	方向角/(°)
1	150	140	243
2	85	85	236
3	150	155	220.5
4	145	50	159
5	130	150	230
新进入	0	0	52

注 3.3 方向角指飞行方向与 x 轴正向的夹角。

3.2.1 建模

对于题目 3.2 的建模和 3.3 相似。使用 Matlab 的非线性规划解决。

3.2.2 代码

#Matlab

```
function [f,g] = fun3_2(x)

    g = [];

    th0 = [243 236 220.5 159 230 52]';
    th = th0 + x;

    x0 = [150 85 150 145 130 0]';
    y0 = [140 85 155 50 150 0]';

    k = 1;

    for i = 1:5

        for j = i+1:6
```

2) 模型二
两架飞机 i, j 不发生碰撞的条件为

$$\begin{aligned} & [x_i(t)-x_j(t)]^2 + [y_i(t)-y_j(t)]^2 \geq 64, \\ & 1 \leq i \leq 5, \quad i+1 \leq j \leq 6, \quad 0 \leq t \leq \min\{T_i, T_j\}, \end{aligned} \tag{3.23}$$

式中: T_i, T_j 分别表示第 i, j 架飞机飞出正方形区域边界的时刻。这里

$$\begin{aligned} x_i(t) &= x_i^0 + at \cos \theta_i, \quad y_i(t) = y_i^0 + at \sin \theta_i, \quad i = 1, 2, \dots, n, \\ \theta_i &= \theta_i^0 + \Delta \theta_i, \quad |\Delta \theta_i| \leq \frac{\pi}{6}, \quad i = 1, 2, \dots, n. \end{aligned}$$

下面我们把约束条件式(3.23)加强为对所有的时间 t 都成立, 记

$$I_{ij} = [x_i(t) - x_j(t)]^2 + [y_i(t) - y_j(t)]^2 - 64 = \tilde{a}_{ij}t^2 + \tilde{b}_{ij}t + \tilde{c}_{ij},$$

式中

$$\begin{aligned} \tilde{a}_{ij} &= 4a^2 \sin^2 \frac{\theta_i - \theta_j}{2}, \\ \tilde{b}_{ij} &= 2a \{ [x_i(0) - x_j(0)] (\cos \theta_i - \cos \theta_j) + [y_i(0) - y_j(0)] (\sin \theta_i - \sin \theta_j) \}, \\ \tilde{c}_{ij} &= [x_i(0) - x_j(0)]^2 + [y_i(0) - y_j(0)]^2 - 64. \end{aligned}$$

则两架 i, j 飞机不碰撞的条件是

$$\Delta_{ij} = \tilde{b}_{ij}^2 - 4\tilde{a}_{ij}\tilde{c}_{ij} \leq 0. \tag{3.24}$$

59

建立如下非线性规划模型:

$$\begin{aligned} & \min \sum_{i=1}^6 (\Delta \theta_i)^2, \\ & \text{s. t. } \begin{cases} \Delta \theta_i \leq 0, & 1 \leq i \leq 5, i+1 \leq j \leq 6, \\ |\Delta \theta_i| \leq \frac{\pi}{6}, & i = 1, 2, \dots, 6. \end{cases} \end{aligned}$$

2016年11月11日 14:14:11

```

        aij = 4 * (sind((th(i) - th(j)) / 2))^2;
        bij = 2 * ((x0(i) - x0(j)) * (cosd(th(i)) - cosd(th(j))) + ...
            (y0(i) - y0(j)) * (sind(th(i)) - sind(th(j))));
        cij = (x0(i) - x0(j))^2 + (y0(i) - y0(j))^2 - 64;
        f(k) = bij^2 - 4 * aij * cij;
        k = k + 1;
    end
end
end

fun3_1 = @(delta) sum(delta.^2); % 定义目标函数的匿名函数

[del,val] = fmincon(fun3_1, rand(6,1), [], [], [], [], ...
    -30 * ones(6,1), 30 * ones(6,1), @fun3_2);

```

#Python

```

import numpy as np
from scipy.optimize import minimize

# 约束函数，对应 MATLAB 的 fun3_2
def constraint_fun(x):
    th0 = np.array([243, 236, 220.5, 159, 230, 52])
    th = th0 + x

    x0 = np.array([150, 85, 150, 145, 130, 0])
    y0 = np.array([140, 85, 155, 50, 150, 0])

    constraints = []
    for i in range(5):
        for j in range(i + 1, 6):
            aij = 4 * (np.sin(np.radians((th[i] - th[j]) / 2)))**2
            bij = 2 * ((x0[i] - x0[j]) * (np.cos(np.radians(th[i])) - np.cos(np.radians(th[j]))) +
                (y0[i] - y0[j]) * (np.sin(np.radians(th[i])) - np.sin(np.radians(th[j]))))
            cij = (x0[i] - x0[j])**2 + (y0[i] - y0[j])**2 - 64
            constraints.append(bij**2 - 4 * aij * cij)
    return np.array(constraints)

# 目标函数，对应 MATLAB 的 fun3_1
def objective_fun(delta):
    return np.sum(delta**2)

# 初始值
x0 = np.random.rand(6) * 60 - 30 # 生成 [-30, 30] 之间的随机数

# 约束
constraints = {'type': 'ineq', 'fun': constraint_fun}

```

```
# 边界
bounds = [(-30, 30)] * 6

# 使用 scipy.optimize 的 minimize 进行优化
result = minimize(objective_fun, x0, method='SLSQP', bounds=bounds, constraints=constraints)

# 输出结果
print("Optimal delta:", result.x)
print("Objective function value:", result.fun)
print("Optimization success:", result.success)
print("Message:", result.message)
```

3.2.3 结果

优化结果:

x 的值 : [-1.38044219e+00 -1.17666356e+00 1.49316877e+00
5.95191380e-05 -1.39763523e+00 6.10961935e-01]
目标函数值: 7.84636950560378
是否成功: True
消息: Optimization terminated successfully

3.3 题目

3.3 用罚函数法求解飞行管理问题的模型二。

例 3.10(本题选自 1995 年全国大学生数学建模竞赛 A 题) 在约 10000m 高空的某边长 160km 的正方形区域内,经常有若干架飞机水平飞行。区域内每架飞机的位置和速度向量均由计算机记录其数据,以便进行飞行管理。当一架欲进入该区域的飞机到达区域边缘时,记录其数据后,要立即计算并判断是否会与区域内的飞机发生碰撞。如果会碰撞,则应计算如何调整各架(包括新进入的)飞机飞行的方向角,以避免碰撞。现假定条件如下:

- (1) 不碰撞的标准为任意两架飞机的距离大于 8km;
- (2) 飞机飞行方向角调整的幅度不应超过 30°;
- (3) 所有飞机飞行速度均为 800km/h;
- (4) 进入该区域的飞机在到达区域边缘时,与区域内飞机的距离应在 60km 以上;
- (5) 最多需考虑 6 架飞机;
- (6) 不必考虑飞机离开此区域后的状况。

请你对这个避免碰撞的飞行管理问题建立数学模型,列出计算步骤,对以下数据进行计算(方向角误差不得超过 0.01°),要求飞机飞行方向角调整的幅度尽量小。

设该区域 4 个顶点的坐标为 (0,0),(160,0),(160,160),(0,160)。飞行记录数据见表 3.7。

表 3.7 飞行记录数据

飞机编号	横坐标 x	纵坐标 y	方向角/(°)
1	150	140	243
2	85	85	236
3	150	155	220.5
4	145	50	159
5	130	150	230
新进入	0	0	52

注 3.3 方向角指飞行方向与 x 轴正向的夹角。

2) 模型二
两架飞机 i, j 不发生碰撞的条件为

$$[x_i(t) - x_j(t)]^2 + [y_i(t) - y_j(t)]^2 \geq 64, \quad 1 \leq i \leq 5, \quad i+1 \leq j \leq 6, \quad 0 \leq t \leq \min\{T_i, T_j\}, \quad (3.23)$$

式中: T_i, T_j 分别表示第 i, j 架飞机飞出正方形区域边界的时刻。这里

$$x_i(t) = x_i^0 + at \cos \theta_i, \quad y_i(t) = y_i^0 + at \sin \theta_i, \quad i = 1, 2, \dots, n,$$
$$\theta_i = \theta_i^0 + \Delta \theta_i, \quad |\Delta \theta_i| \leq \frac{\pi}{6}, \quad i = 1, 2, \dots, n.$$

下面我们把约束条件式(3.23)加强为对所有的时间 t 都成立,记

$$l_{ij} = [x_i(t) - x_j(t)]^2 + [y_i(t) - y_j(t)]^2 - 64 = \tilde{a}_{ij}t^2 + \tilde{b}_{ij}t + \tilde{c}_{ij},$$

式中

$$\tilde{a}_{ij} = 4a^2 \sin^2 \frac{\theta_i - \theta_j}{2},$$
$$\tilde{b}_{ij} = 2a \{ [x_i(0) - x_j(0)](\cos \theta_i - \cos \theta_j) + [y_i(0) - y_j(0)](\sin \theta_i - \sin \theta_j) \},$$
$$\tilde{c}_{ij} = [x_i(0) - x_j(0)]^2 + [y_i(0) - y_j(0)]^2 - 64.$$

则两架 i, j 飞机不碰撞的条件是

$$\Delta_{ij} = \tilde{b}_{ij}^2 - 4\tilde{a}_{ij}\tilde{c}_{ij} \leq 0. \quad (3.24)$$

建立如下非线性规划模型:

$$\begin{aligned} \min \quad & \sum_{i=1}^6 (\Delta \theta_i)^2, \\ \text{s. t.} \quad & \begin{cases} \Delta_{ij} \leq 0, & 1 \leq i \leq 5, i+1 \leq j \leq 6, \\ |\Delta \theta_i| \leq \frac{\pi}{6}, & i = 1, 2, \dots, 6. \end{cases} \end{aligned}$$

Mathematica 求解结果

3.3.1 建模

对于题目 3.3 的建模，注意到这这也是一个非线性规划题目，题目要求最小化所有飞机的转向角度，那么假设每个飞机转向的角度为 $\Delta\theta_i$ ($i = 1,2,3,4,5,6$)目标就是要最小化如下：

$$\min \sum_{i=1}^6 \Delta\theta_i$$

这个就是原始函数，原始的约束如下：

$$\Delta\theta_i \leq 0, 1 \leq i \leq 5, i+1 \leq j \leq 6$$
$$|\Delta\theta_i| \leq \frac{\pi}{6}, i = 1, 2, \dots, 6$$

$$l_y = [x_i(t) - x_i(t)]^2 + [y_i(t) - y_i(t)]^2 - 64$$

罚函数方法根据原始函数和原始约束构建罚函数的目标函数：

$$F(\Delta\theta) = \sum_{i=1}^6 (\Delta\theta_i)^2 + \rho_l \sum_{i=1}^5 \max(0, \Delta\theta_i)$$

3.3.2 代码

```
import numpy as np
from scipy.optimize import minimize

# 目标函数
def fun3_6(t):
    M = 1000
    v = 800

    x0 = np.array([150, 85, 150, 145, 130, 0])
    y0 = np.array([140, 85, 155, 50, 150, 0])
    th = np.array([243, 236, 220.5, 159, 230, 52]) + t

    F = np.sum(t**2)

    g = []

    for i in range(5):
        for j in range(i+1, 6):
            cos_diff = np.cos(np.deg2rad(th[i])) - np.cos(np.deg2rad(th[j]))
            sin_diff = np.sin(np.deg2rad(th[i])) - np.sin(np.deg2rad(th[j]))

            a_ij = v**2 * (cos_diff**2 + sin_diff**2)

            b_ij = 2 * v * ((x0[i] - x0[j]) * cos_diff + (y0[i] - y0[j]) * sin_diff)

            c_ij = (x0[i] - x0[j])**2 + (y0[i] - y0[j])**2
```

```

        if a_ij == 0:
            d_min_sq = c_ij
        else:
            t_min = -b_ij / (2 * a_ij)
            if t_min < 0:
                d_min_sq = c_ij
            else:
                d_min_sq = c_ij - b_ij**2 / (4 * a_ij)
            if d_min_sq < 0: # 避免数值误差
                d_min_sq = 0
        g.append(64 - d_min_sq) # 8^2 = 64

    g = np.array(g)
    penalty = M * np.sum(np.maximum(g, 0))
    max_t = M * np.max(np.abs(t))
    zf = F + max_t + penalty
    return zf

# 初始猜测
t0 = np.zeros(6)

# 约束
bounds = [(-30, 30)] * 6

# 优化
result = minimize(fun3_6, t0, bounds=bounds, method='SLSQP', options={'disp': True})
print("调整量 t (度):", result.x)
print("调整后方向角 (度):", np.array([243, 236, 220.5, 159, 230, 52]) + result.x)
print("目标函数值:", result.fun)

```

3.3.3 结果

Optimization terminated successfully (Exit mode 0)

Current function value: 4014.684703869113

Iterations: 53

Function evaluations: 615

Gradient evaluations: 53

调整量 t (度): [-2.87167819 -2.87983374 3.94321642 -2.8716782
3.94320041 3.94325118]

调整后方向角 (度): [240.12832181 233.12016626 224.44321642 156.1283218
233.94320041

55.94325118]

目标函数值: 4014.684703869113 约束条件检验:

检验碰撞距离，使用程序来检验。

```
import numpy as np

# 调整后方向角
theta_prime = np.array([240.12832181, 233.12016626, 224.44321642, 156.1283218, 233.94320041, 55.94325118])
x0 = np.array([150, 85, 150, 145, 130, 0])
y0 = np.array([140, 85, 155, 50, 150, 0])
v = 800

# 计算每对飞机的最小距离
for i in range(5):
    for j in range(i+1, 6):
        cos_diff = np.cos(np.deg2rad(theta_prime[i])) - np.cos(np.deg2rad(theta_prime[j]))
        sin_diff = np.sin(np.deg2rad(theta_prime[i])) - np.sin(np.deg2rad(theta_prime[j]))
        a_ij = v**2 * (cos_diff**2 + sin_diff**2)
        b_ij = 2 * v * ((x0[i] - x0[j]) * cos_diff + (y0[i] - y0[j]) * sin_diff)
        c_ij = (x0[i] - x0[j])**2 + (y0[i] - y0[j])**2
        if a_ij == 0: # 平行轨迹
            d_min = np.sqrt(c_ij)
        else:
            t_min = -b_ij / (2 * a_ij)
            if t_min < 0:
                d_min = np.sqrt(c_ij)
            else:
                d_min_sq = c_ij - b_ij**2 / (4 * a_ij)
                d_min = np.sqrt(max(d_min_sq, 0))
        print(f"飞机 {i+1} 和 飞机 {j+1} 的最小距离: {d_min:.2f} km")
```

运行代码后，得到以下最小距离：

- 飞机 1 和 飞机 2: 85.15 km
- 飞机 1 和 飞机 3: 15.00 km
- 飞机 1 和 飞机 4: 90.14 km
- 飞机 1 和 飞机 5: 21.54 km
- 飞机 1 和 飞机 6: 205.25 km
- 飞机 2 和 飞机 3: 70.71 km

- 飞机 2 和 飞机 4: 65.00 km
- 飞机 2 和 飞机 5: 53.15 km
- 飞机 2 和 飞机 6: 120.21 km
- 飞机 3 和 飞机 4: 105.12 km
- 飞机 3 和 飞机 5: 20.62 km
- 飞机 3 和 飞机 6: 215.81 km
- 飞机 4 和 飞机 5: 101.98 km
- 飞机 4 和 飞机 6: 153.37 km
- 飞机 5 和 飞机 6: 198.49 km

满足

所有调整量小于 30° ，满足此约束。

3.4 题目

3.4 求下列问题的解：

$$\begin{aligned} \max f(\mathbf{x}) &= 2x_1 + 3x_1^2 + 3x_2 + x_2^2 + x_3, \\ \text{s. t. } &\begin{cases} x_1 + 2x_1^2 + x_2 + 2x_2^2 + x_3 \leq 10, \\ x_1 + x_1^2 + x_2 + x_2^2 - x_3 \leq 50, \\ 2x_1 + x_1^2 + 2x_2 + x_3 \leq 40, \\ x_1^2 + x_3 = 2, \\ x_1 + 2x_2 \geq 1, \\ x_1 \geq 0, x_2, x_3 \text{ 无约束}. \end{cases} \end{aligned}$$

3.4.1 建模

对于题目 3.4 的建模，题目明显提出了一个非线性的规划问题，并且给出了目标函数和约束条件。那么我们可以使用 python 里面的 optimize 库，库中有一个最小化损失函数的方法 `scipy.optimize.minimize`，那么我们将这个最大化目标函数 $f(\mathbf{x})$ 的目的转化为最小化 $-f(\mathbf{x})$ 。也就是我们的目标函数是：

目标函数：

$$\text{minimize } f(x) = 2x_1 + 3x_1^2 + 3x_2 + x_2^2 + x_3$$

约束条件:

$$\begin{cases} x_1 + 2x_1^2 + x_2 + 2x_2^2 + x_3 \leq 10, \\ x_1 + x_1^2 + x_2 + x_2^2 - x_3 \leq 50, \\ 2x_1 + x_1^2 + x_2 + x_3 \leq 40, \\ x_1^2 + x_3 = 2, \\ x_1 + 2x_2 \geq 1, \\ x_1 \geq 0, \quad x_2, x_3 \text{ 不约束.} \end{cases}$$

3.4.2 代码

```
import numpy as np
from scipy.optimize import minimize

# 目标函数 (最大化问题转化为最小化 -f(x))
def objective(x):
    return -(2*x[0] + 3*x[0]**2 + 3*x[1] + x[1]**2 + x[2])

# 约束条件
def constraint1(x):
    return 10 - (x[0] + 2*x[0]**2 + x[1] + 2*x[1]**2 + x[2]) # ≤ 10
def constraint2(x):
    return 50 - (x[0] + x[0]**2 + x[1] + x[1]**2 - x[2]) # ≤ 50
def constraint3(x):
    return 40 - (2*x[0] + x[0]**2 + 2*x[1] + x[2]) # ≤ 40
def constraint4(x):
    return x[0]**2 + x[2] - 2 # 等式约束 x1^2 + x3 = 2
def constraint5(x):
    return x[0] + 2*x[1] - 1 # ≥ 1 (转换为 ≤ -1)

# 变量的边界 (x1 ≥ 0, x2, x3 无约束)
bounds = [(0, None), (None, None), (None, None)]

# 约束定义
constraints = [
    {'type': 'ineq', 'fun': constraint1}, # 不等式约束
    {'type': 'ineq', 'fun': constraint2},
    {'type': 'ineq', 'fun': constraint3},
    {'type': 'eq', 'fun': constraint4}, # 等式约束
    {'type': 'ineq', 'fun': constraint5}
]

# 选择合适的初始点 (根据 MATLAB 参考解)
x0 = np.random.rand(3)

# 求解非线性规划问题, 使用 SLSQP 算法
solution = minimize(objective, x0, bounds=bounds, constraints=constraints, method='SLSQP')

# 输出最优解
print("Optimal solution: x1 =", solution.x[0], ", x2 =", solution.x[1], ", x3 =", solution.x[2])
```

```
print("Maximum value of objective function:", -solution.fun)
```

3.4.3 结果

Optimal solution:

$x_1 = 2.3333333343019577$,

$x_2 = 0.16666663672369514$,

$x_3 = -3.444444489606242$

Maximum value of objective function: 18.08333334334153

约束条件检验:

$$x_1 + 2x_1^2 + x_2 + 2x_2^2 + x_3 \leq 10$$

$2.3333 + 2(2.33332) + 0.1667 + 2(0.16672) + (-3.4445) = 9.9997$ 。满足

$$x_1 + x_1^2 + x_2 + x_2^2 - x_3 \leq 50$$

$2.3333 + (2.33332) + 0.1667 + (0.16672) - (-3.4445) = 11.4094$ 。满足

$$2x_1 + x_1^2 + x_2 + x_3 \leq 40$$

$2(2.3333) + (2.33332) + 2(0.1667) + (-3.4445) = 6.9926$ 。满足

$$x_1^2 + x_3 = 2$$

$(2.3333)^2 + (-3.4445) = 5.4371 - 3.4445 = 1.9926 \approx 2$ 满足

$$x_1 + 2x_2 \geq 1,$$

$2.3333 + 2(0.1667) = 2.3333 + 0.3334 = 2.6667$ 。满足

答案满足约束